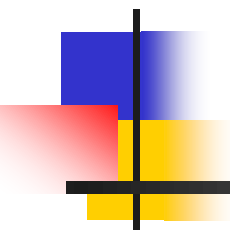


04. Java Class – a new concept



Class fundamentals

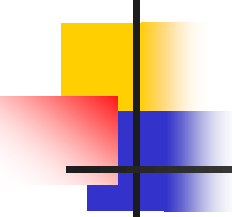
Methods

Constructors

Garbage Collection – Java Way

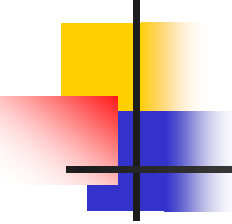
Access Modifiers

Inner Classes



The Class Declaration

<code>public</code>	Class is publicly accessible.
<code>abstract</code>	Class cannot be instantiated.
<code>final</code>	Class cannot be subclassed.
<code><i>cClass NameOfClass</i></code>	<i>Name of the Class.</i>
<code>extends <i>Super</i></code>	Superclass of the class.
<code>implements <i>Interfaces</i></code>	Interfaces implemented by the class.
<pre>{ <i>ClassBody</i> }</pre>	



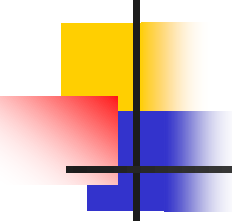
Declaring Member Variables

<code>accessLevel</code>	Indicates the access level for this member.
<code>static</code>	Declares a class member.
<code>final</code>	Indicates that it is constant.
<code>transient</code>	This variable is transient.
<code>volatile</code>	This variable is volatile.
<code>type name</code>	The type and name of the variable.



Details of a Method Declaration

<i>accessLevel</i>	Access level for this method.
<i>static</i>	This is a class method.
<i>abstract</i>	This method is not implemented.
<i>final</i>	Method cannot be overridden.
<i>native</i>	Method implemented in another language.
<i>synchronized</i>	Method requires a monitor to run.
<i>returnType methodName</i>	The return type and method name.
<i>(paramList)</i>	The list of arguments.
<i>throws exceptions</i>	The exceptions thrown by this method.



Example: Stack implementation

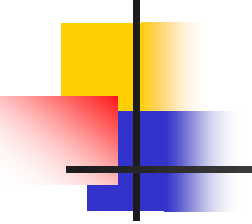
- Implement a class **Stack** with 2 basic operations – *push* and *pop*.

stack

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
32	24	12	8	67					

top





```
class Stack {  
    int stack[] = new int[10];  
    int top;
```

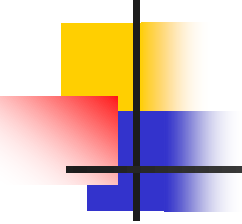
```
    // Initialize top of stack.
```

```
    Stack() {  
        top = -1;  
    }
```

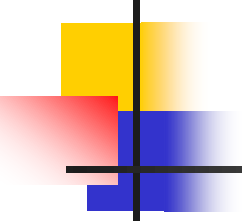
```
    // Push an item on to the stack.
```

```
    void push(int item) {  
        if(top == 9)  
            System.out.println(" Stack is full.");  
        else  
            stack[++top] = item;  
    }
```

// Pop an item from the stack.



```
int pop() {  
    if(top < 0) {  
        System.out.println(" Stack underflow.");  
        return 0;  
    }  
    else  
        return stack[top--];  
}
```



```
public class StackTest {
```

```
    public static void main(String[] args) {
```

```
        // Push some numbers on to the stack.
```

```
        Stack myStack1 = new Stack();
```

```
        for(int i=0; i<10;i++)
```

```
            myStack1.push(i);
```

```
        // Pop these numbers off the stack.
```

```
        System.out.println("Stack in myStack1:");
```

```
        for(int i=0; i<10;i++)
```

```
            System.out.println(myStack1.pop());
```

```
    }
```

```
}
```